

Terminal

Architecture, Data Sources & Open-Source Security Guide

Created by Oscar Zarraga Perez

A technical reference describing how the project collects, caches, and serves market data — written so the source code can be released publicly without exposing credentials or personal information.

Copyright © 2026 Oscar Zarraga Perez · All rights reserved

Document Version: 3.0 — released without warranty.

Disclaimer & Limitation of Liability

No warranty. The Terminal project is provided "AS IS" and "AS AVAILABLE", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, accuracy, completeness, non-infringement, or continued availability of any upstream data source. The project is a personal-use, educational software experiment. Nothing in this document or in the source code creates a contract, an engagement, or a service-level commitment of any kind.

Not investment advice. The data displayed by the project is aggregated from third-party public sources whose accuracy, timeliness, and continuity cannot be guaranteed. The project is not a broker-dealer, investment adviser, financial planner, accountant, auditor, or tax adviser. None of the output of the project, of this document, or of the source code constitutes investment, legal, accounting, tax, or financial advice. Any decision to buy, sell, hold, or evaluate a security is the sole responsibility of the person making that decision and should be taken only after consulting an appropriately licensed professional.

Limitation of liability. To the maximum extent permitted by applicable law, the authors and contributors of the project disclaim all liability for any direct, indirect, incidental, special, consequential, exemplary, or punitive damages — including but not limited to lost profits, lost data, trading losses, missed opportunities, regulatory fines, reputational harm, or any other commercial damage — arising out of or in connection with the use, misuse, or inability to use the project, the documentation, the source code, or any data obtained through the project, even if advised of the possibility of such damages.

Operator responsibility. Anyone who downloads, runs, modifies, redistributes, or deploys the project (the "operator") is solely responsible for: (a) compliance with the terms of service of every upstream data source the project queries; (b) compliance with all applicable laws, including securities, market-data, privacy, export, and tax laws in the operator's jurisdiction; (c) the security of any API keys, credentials, or other secrets the operator chooses to configure; (d) any consequences arising from public, commercial, or multi-user deployments of the project.

Third-party services. The project queries third-party endpoints. It does not control, endorse, or warrant any third-party content, service availability, accuracy, or terms. Endpoints may change, rate-limit, or cease without notice. Trademarks, service marks, and logos referenced in this document are the property of their respective owners; references are nominative and do not imply endorsement, affiliation, or sponsorship in either direction.

By using the project in any form, the operator acknowledges and accepts these terms in full.

Technical Documentation

Architecture, every data source, every endpoint, the in-app tab proxy, the cache and retry strategy, the multi-layer garbage detector, the unified price resolver, and the build history from two separate Phase-1 terminals into one unified application. Companion to *SETUP.pdf* (install + quick-start).

Document Version 3.0 — Copyright © 2026 Oscar Zarraga Perez · MIT License.

1. Architecture

The terminal is intentionally minimal in dependencies. The entire backend is one Python file (*markets_data_api.py*, ~9,500 lines) that uses only the standard library — no *pip install* required. It boots an HTTP server on port 8787 that does three things:

- Serve the static HTML pages (*terminal.html* shell + four sub-pages).
- Aggregate data from public APIs, RSS feeds, and HTML scrapes, and expose them as */api/** JSON endpoints.
- Proxy external pages through */api/proxy?url=...* so SEC filings, press releases, and news articles can be embedded inside the terminal as new in-app tabs without spawning a Chrome window.

Page topology

terminal.html is the shell — a tab manager with a top tab bar. It pre-loads three pinned tabs as iframes pointing at the three sub-pages, and listens for *postMessage* events. When a sub-page wants to open an external link, it posts *{type:'unified-terminal:open-tab', url, title}* — the shell creates a new iframe whose *src* is */api/proxy?url=<target>*.

Tab	Sub-page	Primary endpoints
Markets	markets.html	<i>/api/quote</i> , <i>/api/prices</i> , <i>/api/chart</i> , <i>/api/summary</i> , <i>/api/news-all</i> , <i>/api/press</i> , <i>/api/sec/*</i>
Watchlist Feed	calendar.html	<i>/api/news-all/<sym></i> , <i>/api/press/<sym></i> , <i>/api/summary/<sym></i> , <i>/api/earnings/rich-batch</i>
Policy Watch	policy.html	<i>/api/policy/today</i> (fans out to 10 free sources in parallel)

The unified price resolver — `resolve_prices()`

The watchlist endpoint */api/quote* and the dedicated */api/prices* endpoint both delegate to a single resolver that walks every available source in reliability order, stopping the moment a price returns. Layered, run in parallel where possible:

- **Layer 1 — CoinGecko batch:** one HTTP for every crypto symbol in the request.
- **Layer 2 — Yahoo v8 chart, parallel:** works for stocks/indices/FX/futures. **Skipped for crypto** (Yahoo's v8 has a degraded mode where it echoes back the wrong symbol's data — SOL-USD coming back with BTC's price). Also rejects any response whose *meta.symbol* doesn't match.
- **Layer 2a — Binance public ticker:** covers any crypto CoinGecko missed; no key.

- **Layer 2b — MarketWatch index scrape:** dedicated source for indices, parses `<bg-quote field="Last|Change|PercentChange">` from the `intraday__data` block.
- **Layer 2c — CNBC quote API:** `quote.cnbc.com/quote-html-webservice/restQuote`; free JSON for the same indices.
- **Layer 3 — Twelve Data batch:** with `:INDX` exchange suffix for indices and `EUR/USD` format for FX.
- **Layer 4 — frankfurter.app:** USD-based FX rate table, one HTTP gets every requested pair. Replaces the now-defunct `exchangerate.host`.
- **Layer 5 — Finnhub:** per-stock fallback for anything Yahoo missed.

Independent of the resolver, the `/api/quote` handler also has a **fast path for all-crypto requests**: when every requested symbol is crypto, the handler skips the entire stock pipeline (Yahoo v7 batch + Finnhub + FMP + Stooq) and goes straight to the resolver. Round-trip for a single crypto symbol is typically under 300 ms.

2. Data sources

Every source the terminal uses, broken down by what it is for. All sources are free; the four key-protected providers in the second table are optional — the terminal works fully without any keys.

Free / no key required

Source	What it provides	Endpoint base
Yahoo Finance v7	Batch quotes for stocks/indices/FX/commodities	query1.finance.yahoo.com
Yahoo Finance v8	OHLCV charts, single quotes, meta	query1.finance.yahoo.com/v8
Yahoo HTML / RSS	News, press releases, SEC filings list	finance.yahoo.com
Stooq	CSV quotes + history fallback for ~everything	stooq.com/q/l/
CoinGecko	BTC + crypto price, market cap, dominance, history	api.coingecko.com/api/v3
Binance	Crypto ticker fallback (24h price/change/volume)	api.binance.com/api/v3
MarketWatch	Index scrape via <code><bg-quote field=...></code>	marketwatch.com/investing/index
CNBC quote API	Free JSON quote service for major indices	quote.cnbc.com
frankfurter.app	ECB-sourced FX reference rates (USD base)	api.frankfurter.app
open.er-api.com	Backup FX rate table (USD base)	open.er-api.com/v6
mempool.space	Block height, hash rate, mempool, fees, Lightning	mempool.space/api
alternative.me	Bitcoin Fear & Greed Index	api.alternative.me/fng

Source	What it provides	Endpoint base
SEC EDGAR	10-K, 10-Q, 8-K filings; XBRL facts; Form 4	data.sec.gov / www.sec.gov
Nasdaq public	Earnings/economic/IPO/dividend/splits calendar	api.nasdaq.com/api/calendar
Federal Register API	SEC/CFTC/Fed/Treasury rules + comment deadlines	federalregister.gov/api/v1
Fed/SEC/CFTC/Treas RSS	Press releases, FOMC, OFAC actions	federalreserve.gov, sec.gov
Committee RSS	Senate Banking + Ag, House Fin Svcs + Ag hearings	banking.senate.gov, etc.
ECB / BoE RSS	International central bank press	ecb.europa.eu, bankofengland.co.uk
Farside	Daily BTC + ETH spot ETF net flows	farside.co.uk
Google News RSS	Symbol-keyed news aggregation (decoded)	news.google.com/rss/search
MarketWatch RSS	Symbol-filtered news stream	feeds.content.dowjones.io
Finviz	Snapshot scrape (52w stats, earnings date)	finviz.com/quote.ashx
StockAnalysis.com	Fundamentals + intl ADR coverage	stockanalysis.com

Optional keyed providers (free tiers)

Provider	What it adds	Free quota
Finnhub	Real-time stock quotes; deep metrics (P/E, beta, 52w H/L, EPS, dividend, market cap, shares out)	60 req/min
FMP	Wider equity universe inc. ADRs/intl; per-quote market cap; batch endpoint	250 req/day
Twelve Data	Index quotes via :INDX suffix; FX via EUR/USD format; batch endpoint	8 req/min · 800/day
Alpha Vantage	Optional cross-check for stocks/FX	5 req/min · 500/day

3. API endpoints

Markets / quotes / fundamentals

Endpoint	Returns
GET /api/healthz	Server health check
GET /api/keys/status	Which keyed providers are configured
GET /api/quote?symbols=A,B,C	Yahoo-shape batch quote; routed by ticker type, final resolve_prices() pass for misses
GET /api/prices?symbols=A,B,C	Unified resolver, returns {prices,count,requested,missing}
GET /api/summary/<sym>	Full quoteSummary: price, profile, financials, SEC filings
GET /api/chart/<sym>	Yahoo-shape OHLCV chart payload
GET /api/financials/<sym>	Income / balance / cash flow (annual + quarterly)
GET /api/statistics/<sym>	Key statistics (P/E, beta, etc.)
GET /api/analyst/<sym>	Price targets, ratings, EPS forecast, surprise history
GET /api/insider/<sym>	SEC Form 4 insider transactions
GET /api/holders/<sym>	Top institutional holders (Nasdaq)
GET /api/dividends/<sym>	Dividend history
GET /api/earningsdate/<sym>	Next earnings date + consensus EPS
GET /api/earnings/rich-batch?symbols=...	Batch rich-earnings (parallel server-side)
GET /api/options/<sym>	Yahoo-shape options chain

News / press releases / filings

Endpoint	Returns
GET /api/news-all/<sym>	Aggregated news (Yahoo Finance RSS primary + Google News, Google URLs decoded to publisher URLs)
GET /api/press/<sym>	Press releases (SEC 8-K 99.1, BusinessWire, GlobeNewswire, PRN, Yahoo)
GET /api/sec/filings/<sym>	SEC EDGAR filing list with direct links
GET /api/sec/insider/<sym>	Form 4 insider transactions
GET /api/sec/facts/<sym>	Company-facts XBRL

Policy Watch + Calendar + In-app proxy

Endpoint	Returns
GET /api/policy/today	Aggregated payload from 10 free sources in parallel
GET /api/policy/fed-register	Federal Register documents + comment deadlines

Endpoint	Returns
GET /api/policy/etf-flows?asset=...	Farside daily ETF net flows scrape
GET /api/policy/links	The 54-link curated directory
GET /api/calendar/earnings?date=...	Earnings reports scheduled on that date
GET /api/calendar/economic?date=...	Economic events (CPI, NFP, FOMC, etc.)
GET /api/calendar/ipos?date=YYYY-MM	Monthly IPO calendar (priced + filed + upcoming)
GET /api/proxy?url=...	Server-side fetch + strip frame headers + base-href rewrite + Google News URL decode + SEC EDGAR landing render

4. How the in-app tab proxy works

When the user clicks an SEC filing or a news article inside the terminal, we do not want a Chrome window to pop. Instead, the click is intercepted and routed through `/api/proxy?url=<target>`, which:

- **Picks request headers per host.** SEC.gov demands a contact email in the User-Agent (the project ships with *Unified-Terminal contact@example.com*); Google domains get a CONSENT cookie so the EU interstitial doesn't fire; everything else gets a realistic browser UA with Sec-Fetch hints and an *Accept-Encoding: gzip, deflate* the proxy decompresses on the way out.
- **Decodes Google News article URLs in flight.** The `/articles/<token>` path contains a URL-safe base64-encoded protobuf with the publisher URL inside. `_decode_google_news_url()` extracts it; the proxy fetches the publisher article instead of Google's interstitial.
- **Intercepts SEC EDGAR landing URLs.** The modern SPA at `/edgar/browse/?CIK=...` can't boot inside our sandboxed iframe. `_render_sec_edgar_landing()` calls `data.sec.gov/submissions/CIK<cik>.json` directly and renders a clean filings page with company facts and the 60 most recent filings linked to their actual documents.
- **Performs the GET** with `urllib`, follows redirects, captures the final URL.
- **Rewrites the HTML:** injects `<base href>` from the final URL so relative CSS/images resolve; strips inline `<meta http-equiv="content-security-policy">`, `x-frame-options`, and `refresh` tags; injects a CSS reset so the page renders on white even if its stylesheet fails inside the iframe; injects a click-interceptor that catches link clicks and form submits and pops them as new in-app tabs.
- **Sends back** with *X-Frame-Options: ALLOWALL* and *Content-Security-Policy: frame-ancestors ** so the parent shell can embed it.

5. Reliability layers

Provider separation

The watchlist (Markets) and the Policy Watch / Watchlist Feed tabs are deliberately routed through different primary providers so they cannot drain each other's rate-limit budgets. Markets primary: Yahoo v7 batch → Finnhub backup. Watchlist Feed primary: news/press aggregators (mostly RSS). Policy Watch primary: Federal Register API + agency RSS.

Caches

Module-level cache (`_cache_get` / `_cache_put`) with per-key TTL. Used by every helper that hits the network. TTLs range from 10 seconds for stock quotes up to 24 hours for SEC company facts. Per-source soft cache for the Bitcoin Intel endpoint and the Policy Watch helpers; each source returns last known good if the refetch fails — it never overwrites real data with empty data. Watchlist row cache keyed by sorted symbol-set, with a >50%-jump rejector so a single bad Yahoo response can't blank a row.

Multi-pass garbage detector (Yahoo v7 batch)

Three independent checks; if any fires, the whole batch is rejected and the caller falls through to the next provider:

- Exact-duplicate >50% — more than half the returned rows share a single *regularMarketPrice*.
- Cluster $\geq 40\%$ — at least 40% of the returned rows have prices within 1% of the median (catches the "all stocks around \$61,000" failure where Yahoo silently propagates BTC's price).
- Per-row sanity — any single row whose price differs from its own *regularMarketPreviousClose* by more than 50% (clear sign of cross-contamination).

Yahoo HTML scraper safeguards

The *finance.yahoo.com/quote/<sym>* page embeds multiple *QuoteSummaryStore* blocks — one for the main symbol and one per sidebar/related ticker. The scraper walks every block and prefers the one whose *price.symbol* matches the requested ticker; if none match by symbol it accepts the first store that doesn't **explicitly disagree** (i.e. has no symbol field or matches case-insensitively). Only rejects when every QSS on the page carries an explicit different symbol — the original BTC-on-AAPL contamination case.

MarketWatch index scraper safeguards

MarketWatch pages always start with a "Markets Overview" strip listing the major US indices, so the first *<bg-quote field="Last">* on every page would be the DJI from that strip — not the page's symbol. The scraper scopes its regex to the *<div class="intraday__data|__price|__column">* block (the only one containing the page's own symbol), with JSON-LD structured-data as a fallback.

Stale-while-error

Every per-source fetcher in the Bitcoin Intel and Policy Watch paths is wrapped in *_src_cached_or_fetch(name, fetcher)*. Cache hit → return; cache miss → call fetcher; fetcher exception or empty return → serve last known good value. Net effect: prices and stats never blank out — they go stale and stay visible.

Concurrency / parallel fan-out

Where many independent network calls are needed (BTC Intel sub-fetches, Policy Watch aggregator, basket quotes, non-stock side panels, rich-earnings batch, every layer of *resolve_prices()*), the code uses *ThreadPoolExecutor(max_workers=8)*. Wall-time becomes max(call time), not sum.

6. Build history

The unified terminal evolved from fusing two separate Phase-1 codebases plus a Phase-2 refactor focused on reliability:

Milestone	What it added
Unified shell (terminal.html)	Top tab bar + iframe stack + postMessage routing
In-app tab proxy	/api/proxy with frame-header stripping + base href rewriting
SEC custom landing renderer	_render_sec_edgar_landing() bypasses the broken SPA
Google News URL decoder	_decode_google_news_url() resolves /articles/<token> to publisher URL
Multi-pass garbage detector	Exact dup + cluster + price-vs-prev-close checks on Yahoo v7
Lenient Yahoo HTML walker	Picks the QSS that matches the requested symbol; explicit-mismatch reject only
MarketWatch index scraper	Dedicated free source for ^GSPC / ^DJI / ^IXIC / ^FTSE / ^GDAXI / ^HSI
CNBC quote API	Second independent index source via quote.cnbc.com
frankfurter.app FX	Replaced now-paywalled exchangerate.host
Twelve Data fallback	Index/FX coverage via :INDX suffix and EUR/USD format
Binance crypto fallback	When CoinGecko stutters; via ticker/24hr endpoint
Unified /api/prices endpoint	Single resolver, walks every layer, parallel where possible
Crypto fast path in /api/quote	All-crypto requests skip the whole Yahoo pipeline
Race-condition guard in markets.html	_LOAD_TOKEN discards stale in-flight responses
Tab visibility logic	Indices/FX/crypto show only NEWS; futures + stocks keep their tabs
Silent Windows launcher	start_windows.vbs runs python with no console window
Brighter UI palette	--text/--muted/--dim raised to #fff/#e8e8e8/#b8b8b8 for readability

7. File structure

Everything required to deploy is in the project folder. The Python file is the only non-static piece; the HTML files are vanilla and have no build step:

File	Purpose
markets_data_api.py	Backend HTTP server. ~9,500 lines, stdlib only. Boots on port 8787.
terminal.html	Shell page. Tab manager + postMessage bus. Served at /
markets.html	Markets tab UI (quote desk + watchlist + side panels + bottom tabs).
calendar.html	Watchlist Feed tab UI.

File	Purpose
policy.html	Policy Watch tab UI.
bitcoin.html	Legacy Bitcoin Intel page (still served).
start_mac_linux.command	Mac / Linux launcher (bash).
start_windows.bat	Windows launcher (delegates to .vbs).
start_windows.vbs	Silent Windows launcher (no console window).
keys.json.example	Template for the optional API keys file.
LICENSE	MIT, Copyright © 2026 Oscar Zarraga Perez.
README.md / CHANGELOG.md	Project overview + change history.
Terminal Architecture.pdf / SETUP.pdf	This document + companion setup guide.
.gitignore	Excludes keys.json, __pycache__, .env, IDE folders.

8. License & attribution

Released under the MIT License. **Copyright © 2026 Oscar Zarraga Perez.** The Phase-1 base codebases (Markets Data Reuse Pack and Bitcoin Intelligence Terminal) were public-domain Unlicense. Free / public data sources retain their own copyrights; this terminal does not redistribute their data, only fetches and displays it on the user's behalf.

© 2026 Oscar Zarraga Perez · MIT License · See *SETUP.pdf* for install instructions.